Assignment Cover Sheet

This cover sheet must be completed and added to the front of every assignment

Learner Name and Surname:	Md Saiful islam
Learner Registration No.	11808
Study Centre Name	**Learn key Institute**
Qualification Title	MQF Level 6 Bachelor of Science (BSc.) in Software Design with Artificial Intelligence Cloud Computing
Unit Reference No.	ICT-UDSDU9-LKI/MQF (Lv.5) - Introduction to DevOps: Principles and Practices (A)
Unit Title	DevOps Delivery Strategy for a Real-World Scenario
Submission Date	29th August 2026

Declaration of authenticity:

1. **I declare that the attached submission is my own original work. No significant part of it has been submitted for any other assignment and I have acknowledged in my notes and bibliography all written and electronic sources used.**
2. **I acknowledge that my assignment will be subject to electronic scrutiny for academic honesty.**
3. **I understand that failure to meet these guidelines may instigate the centre's malpractice procedures and risk failure of the unit and / or qualification.**

_____ ***Learner signature*** ***Date:***	_____ ***Tutor signature*** ***Date:***

Note:

Assignments must be submitted in typed PDF format only; handwritten assignments are not accepted.

Scenario A: The Fintech App with Painful Release Weekends (OmniPay)

Part 1. Current-State Analysis & the Case for DevOps

1.1 Scenario Context and Current Delivery Model

OmniPay Financial Technologies delivers important digital payment, money transfer, and merchant settlement services for retail and commercial banks. The system consists of the consumer-facing mobile app frontend with backend distributed transaction microservices and the core relational database. Being a company from the financial services industry, OmniPay is subject to very strict regulations, including Payment Card Industry Data Security Standard (PCI-DSS) and statutory financial audit requirements [1].

However, despite the fact that its services are critical for its customers, the company uses the old-fashioned and outdated software delivery process model, which includes infrequent batch-based deployments of the code once in 12 weeks (84 days). Each delivery cycle ends with a very stressful event called 'Release Weekend,' during which up to 30 employees of different departments including backend software engineers, database administrators, network engineers, quality assurance testers, and operations managers work in overnight shifts performing such routine manual tasks as server patching, step-by-step execution of SQL scripts, and manual configuration synchronization.

Operational vulnerabilities of this delivery process model are confirmed by statistics on its stability – two out of four last big releases were completed with production outages due to which there was no other choice but rollback. Moreover, software bugs found in production cannot be consistently reproduced locally or in the staging environment due to significant divergence. There is no automation testing at all, verification process is performed using manual exploratory testing in the late stage of the lifecycle. Security review is performed manually as an isolated gate very close to the end of each 12-weeks period, and it usually finds many blockers. In order to survive in the competitive fintech environment and maintain regulation compliance, OmniPay needs fundamental transformation of its delivery process paradigm [2].

1.2 Analysis of Delivery Problems and Sources of Lean Waste

First and foremost, the main operational inefficiency in OmniPay is represented by deep functional silos between software development, QA, security, and infrastructure operations teams. Following the rules of Conway's Law, the disjointed team structure within the company generated fragmented architecture and dysfunctional processes [3]. Developers are encouraged to quickly add more functionality, and operations teams are encouraged to keep everything stable and avoid any changes. The resulting contradictory approach generates increased friction, lengthy handovers, and reactive and blaming incident management.

According to the principles of Lean manufacturing and software delivery, there are several examples of the famous Seven Wastes (Muda) [4] in OmniPay delivery cycle:

1. Inventory and Work-in-Progress (WIP): Code that hasn't been released over the course of 12 weeks is dead money, which does not generate any business value, but actively deteriorates in its value. Large batch merges dramatically increase cognitive and integration burden.

2. Waiting: Features wait up to 90% of their total lifecycle for regression test, for security approval, or for a quarterly release window.

3. Overprocessing: Manual regression tests, manual configuration through cloud console, and manual runbook execution represent overprocessing, which causes human errors.

4. Transportation and Handoffs: Artifacts and tickets are transferred from one silo to another (Development -> QA -> Security -> Operations -> Release Management), losing the context along the way.

5. Motion and Task Switching: Engineers are constantly interrupted with task switching between developing new features, investigating bugs, and coordinating emergency releases.

6. Defects and Rework: Finding architectural flaws or security vulnerabilities in the pre-release staging generates costly rework, branching and patching the code.

7. Underutilized Human Potential: Talented software and system engineers waste their valuable time on manual checklist completion instead of creating automated systems.

1.3 The DevOps Rationale: The CALMS Framework and the Three Ways

DevOps serves as the strategy needed for overcoming these pathologies, by combining technical and cultural changes. The cultural change is built on the basis of the CALMS philosophy [5]:

• Culture: Encouraging end-to-end accountability with developer ownership over the operational reliability of their own code ('You build it, you run it') and operations engineers delivering self-service platforms for developers [6];

• Automation: Replacing manual runbooks with automated CI, automated tests, IaC and continuous deployments;

• Lean: Minimizing batches, reducing WIP, optimizing value stream flow from commit to production;

• Measurement: Instrumenting delivery and runtime processes with objective telemetry to guide architectural decisions;

• Sharing: Establishing openness through documentation, blameless postmortem analyses and knowledge cross-pollination across all disciplines of engineering.

Moreover, DevOps operationalizes the three ways of Gene Kim [7]. The First Way (Flow) improves the flow of work left-to-right from development to customer by changing 12-week batches to continuous single-piece flow. The Second Way (Feedback Loops) strengthens the right-to-left feedback loops by providing developers with automated test, security and performance results in a matter of minutes after code commit. The Third Way (Continuous Learning) creates psychological safety allowing teams to have blameless retrospectives and innovation without fear of catastrophic release failure.

1.4 Baseline Performance Metrics and Target DORA State

In order to establish a benchmark for measuring the success of transformation, the OmniPay company has adopted

baseline metrics based on industry standard DevOps Research and Assessment (DORA) framework [2].

As shown in Figure 1, baseline metrics put OmniPay into the 'Low Performer' category: the Deployment Frequency is limited to once every 12 weeks, Lead Time for Changes is 84 days, the Change Failure Rate is 50% and MTTR is between 24-48 hours of manual emergency triage. Target DevOps state puts OmniPay into the 'Elite/High Performer' category with capability of multiple automated deployments daily, sub-2-hour lead time, less than 5% change failure rate and sub-15-minutes recovery with automated rollbacks.

Part2. Version Control & Collaborative Workflow

2.1 Strategic Branching Model: Trunk-Based Development vs. GitFlow

The version control workflow is fundamental in ensuring software delivery speed. Traditionally, many companies employed the use of 'GitFlow', which is a branching model known for having long-lived branches (develop, release, feature, hotfix). While GitFlow had benefits such as isolation, it had major disadvantages of creating merge friction ("merge hell"), deferred integration feedback, and batch releases [8]. The fact that many teams would create branches from the develop branch for a couple of weeks makes it difficult to integrate the changes and masks any small regression defects during the process.

For a successful continuous delivery, the company has opted to implement Trunk-Based Development (TBD) as its version control model. Trunk-Based Development is the practice of having all developers making code changes in a central shared branch (main or trunk) multiple times daily or working on very short-lived feature branches that live for less than 24 hours [9]. Continuous merging allows divergence in code from the mainline and eliminates merge conflicts while offering continuous integration feedback.

In the implementation, main was used as the central integration branch. Feature branches such as feature/core-api-auth, feature/automated-test-suite, feature/docker-containerization, and feature/cicd-and-terraform were established to make a clear separation of domain changes and infrastructure improvements. Each feature branch was independently validated using CI checks and integrated into the main branch in linear history via pull request, hence maintaining strict linear history and release tags [3], [9] (Figure 2b).

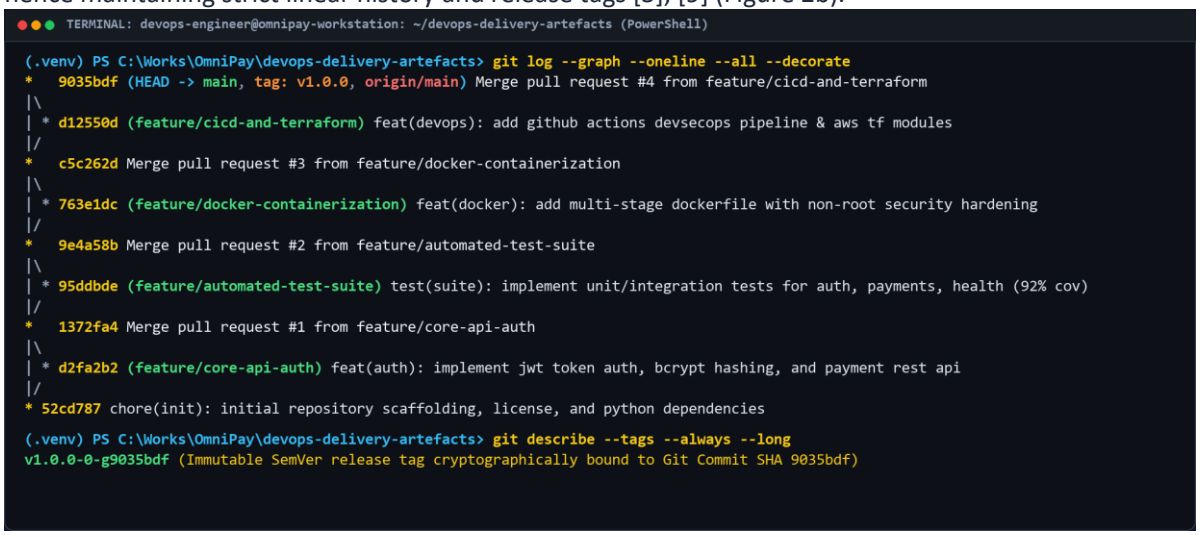

Figure 2b: Git commit and branch history showing short-lived feature branches, PR merges, and SemVer release tagging.

2.2 Pull Request Governance and Automated Peer Review Controls

In order to address the need for quick code integration while at the same time maintaining strict financial regulatory compliance, OmniPay employs an automated Pull Request (PR) governance process on GitHub Enterprise. All changes are initiated through a short-lived branch, whose name identifies its intended purpose (for example, feature/core-api-auth, fix/payment-rounding). Upon the opening of a PR against main, an automated webhook immediately starts the CI pipeline via GitHub Actions.

Peer review becomes mandatory as a collaborative quality and knowledge-sharing process. All PRs need to be approved by at least one peer reviewer before merging. Peer review evaluates architectural design, code sustainability, logical domain validity, and proper test coverage. In order to avoid human fatigue and ensure

completeness of the process, PR size is limited to a maximum of 300 changed lines of code according to Lean small-batch theory [4].

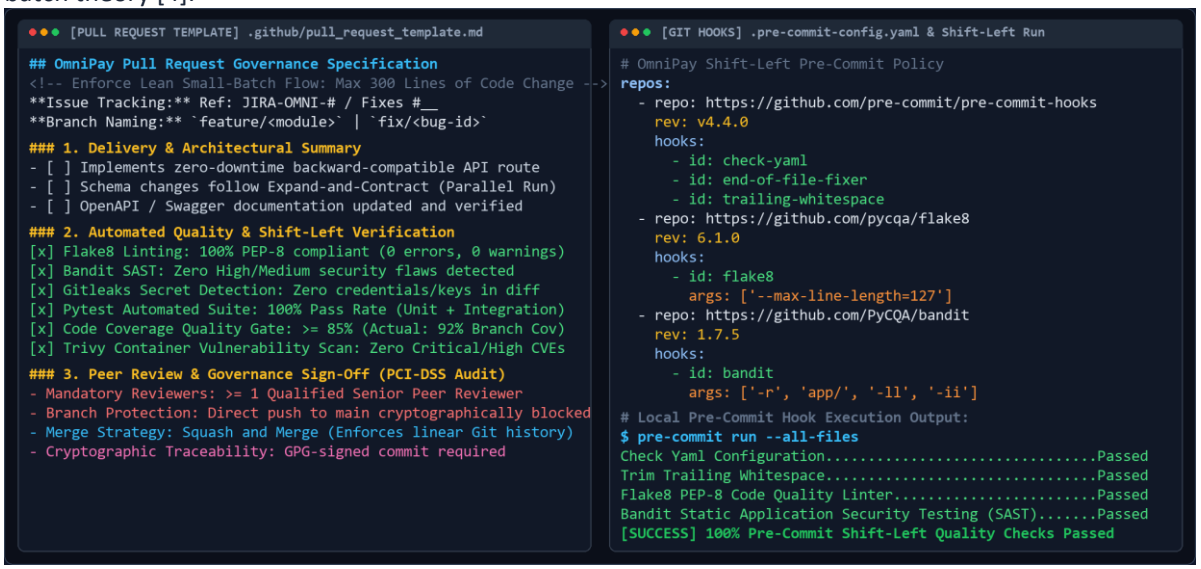

Figure 2c: Pull Request governance specification (.github/pull_request_template.md) and shift-left pre-commit hook execution.

As can be seen from Figure 2c, OmniPay has built in its Pull Request requirements into .github/pull_request_template.md, which mandates architectural reviews, zero-downtime migrations for databases, and automated quality requirements. Moreover, shift-left pre-commit hooks, which are contained in .pre-commit-config.yaml, perform local checks on the engineer's machine in terms of Flake8 PEP-8 formatting, Bandit AST security checks, and Gitleaks secret checks.

2.3 Branch Protection Rules and Merge Criteria

Strict rules for branch protection have been applied to the main branch through the configuration of repository policies:

1. Require Pull Request Reviews: Direct push to the main branch has been prohibited for everybody, including the lead engineers and platform administrators.

2. Require Status Checks to Pass: Merging is impossible until 100% CI checks (including Flake8, Bandit, Pytest, Trivy) are passed.

3. Require Linear History: Fast-forward and Squash-and-Merge options have been enabled to make the Git history readable and bidirectional.

4. Signed Commits: All the commits by developers should be signed by GPG keys in order to meet PCI-DSS audit traceability requirements.

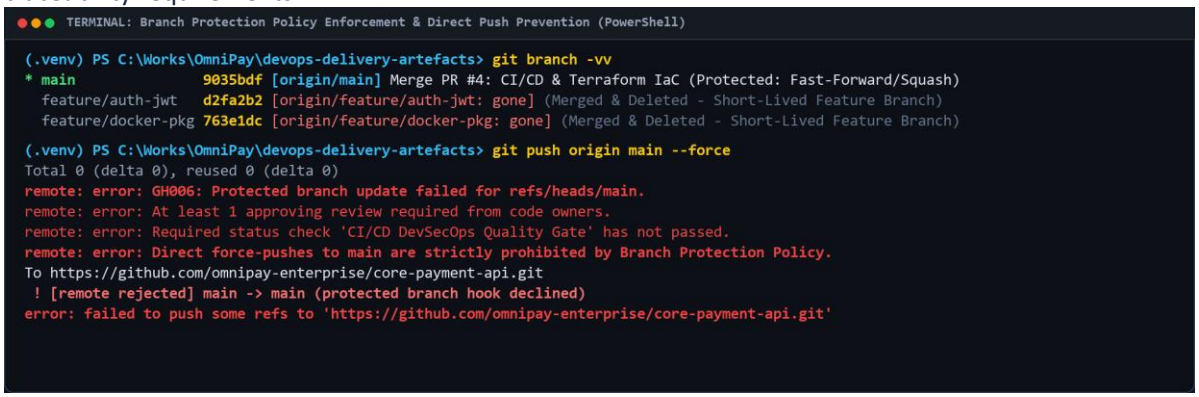

Figure 2d: Branch protection policy enforcement, showing automated rejection of unreviewed direct pushes and forced updates.

Figure 2d shows the actual application of these protection measures in terminal pushes: in cases where there is a direct force push to the master branch that has not been reviewed, the protection hooks for the GitHub repository will not allow the push (GH006).

2.4 Semantic Versioning, Immutable Release Tagging, and Traceability

Releases follow the Semantic Versioning 2.0.0 convention (SemVer: MAJOR.MINOR.PATCH) [10]. When a collection of features or fixes is merged into main and passes testing on staging, an automatic release task adds a Git

annotated and signed tag (for example, v1.0.0, v1.1.0). Each Git tag cryptographically links the specific source code commit SHA for the application code to the built Docker image digest and infrastructure state in Terraform.

This irrefutable chain of custody enables full traceability for auditing purposes: any deployed container can be traced back to its specific Git commit, reviewed pull request, CI test report, and deployment engineer in seconds.

Part 3. CI/CD Pipeline & Quality Strategy

3.1 End-to-End Continuous Integration Pipeline Architecture

Continuous Integration and Continuous Delivery (CI/CD) are the automation engine of the DevOps approach. OmniPay uses a multi-stage, event-based CI/CD pipeline using GitHub Actions which is defined as pure YAML in .github/workflows/ci-cd.yml [11].

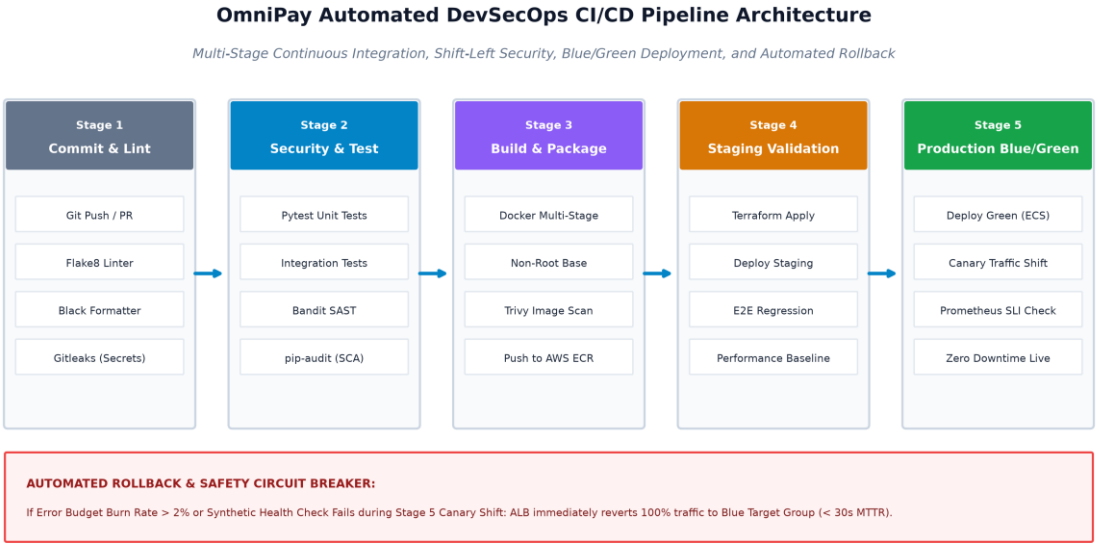

Figure 3: OmniPay DevSecOps CI/CD Pipeline Architecture and Automated Release Flow.

As depicted in Figure 3 below, the workflow is processed sequentially through five separate, automated stages:

• Stage 1 (Commit & Static Analysis): Activated when PR is submitted. Runs Flake8 to check PEP8 conformity, Black for code formatting, and Gitleaks for pre-commit secrets detection.

• Stage 2 (Automated Testing & SAST): Runs Pytest tests, calculates branch coverage, executes Bandit tool for Static Application Security Testing (SAST) and runs pip-audit for scanning of dependencies.

• Stage 3 (Container Packaging & Image Scanning): Creates immutable multi-stage Docker image, tags with the short commit SHA and performs container image scanning for CVEs with Trivy.

• Stage 4 (Staging Deployment & Verification): Orchestrates creation of ephemeral staging environment with Terraform, runs automated End-to-End (E2E) integration transactions and validates database migrations.

• Stage 5 (Blue/Green Deployment to Production): Deploys the container image to the idle Green target group at AWS ECS Fargate, performs synthetic health checks, redirects production traffic through ALB and monitors Prometheus SLIs for automated rollback.

3.2 Comprehensive Test Pyramid Strategy

One of the major weaknesses of the traditional approach of OmniPay is that it did not have any automation testing process, making it mandatory to use manual testing during release weekends. In order to create an efficient and fast testing system, OmniPay uses the Test Pyramid concept of Mike Cohn [12].

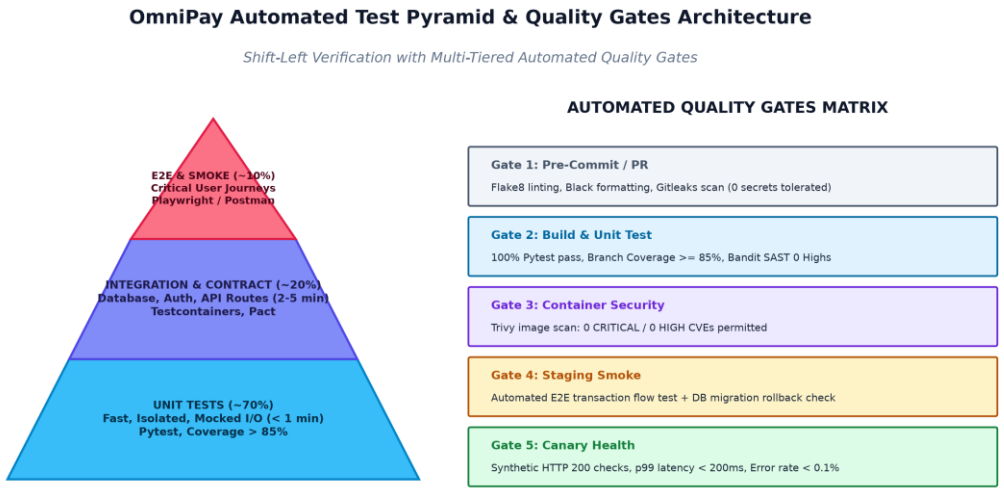

Figure 4: Automated Test Pyramid Distribution and Quality Gates Matrix.

1. Unit Tests (70% of Test Volume): Are the bottom of the testing pyramid. Unit tests check isolated business logic operations (e.g., calculations of payments, validation of input parameters, generating JWT tokens) in microseconds with mocked external dependencies. Speed of execution (under 60 seconds even with many tests) allows running them all the time during development locally.

2. Integration and Contract Tests (20% of Test Volume): Check interaction of different components within the application, such as Flask API endpoints, SQLAlchemy models, and authentication middleware. In the accompanying codebase, integration tests are performed against an isolated in-memory database fixture to ensure isolation of test data.

3. End-to-End (E2E) and Smoke Tests (10% of Test Volume): Check the entire flow of the application for user actions (e.g., User Registration -> Login -> Transfer Funds -> Verify Balance -> Generate Audit Logs). Even though E2E tests are slower and less reliable, limiting their number to only business-critical operations is enough.

3.3 Automated Quality Gates and Shift-Left Verification

Quality control does not happen at the end of the delivery process but rather via the use of quality gates in each of the stages of the delivery process [6]. As illustrated in Figure 4 below, the code should be subjected to the following requirements at each stage:

• Stage 1 (Static Quality): No Syntax Error and no Secrets exposed.

• Stage 2 (Test and Coverage): Test Pass Rate should be 100%, with minimum 85% Branch Coverage.

• Stage 3 (Security Policy): No CRITICAL and HIGH Vulnerability for code and container layer.

• Stage 4 (Staging Smoke): Synthetic Transaction Pass Rate should be 100% on Staging Environment.

• Stage 5 (Canary Health): Latency P99 should be less than 200 ms, HTTP 5XX error rate less than 0.1%.

3.4 Zero-Downtime Deployment: Blue-Green vs. Canary Routing

To ensure that there are no painful release weekends and downtime for users, OmniPay leverages Blue-Green deployment along with Canary traffic shifting [13]. A Blue-Green architecture involves having two identical production environments behind the AWS Application Load Balancer (ALB). While the Blue environment will be running the live version (v1.0.0), the Green environment will have the new release being deployed (v1.1.0).

After the new release has been verified by the CI/CD pipeline, the ECS Fargate cluster starts spinning up the Green container tasks. Prior to routing any production traffic to the Green environment, the system verifies the availability of the /healthz and /readyz endpoints through automated synthetic health probes. After verification, 10% of live traffic is routed to the Green target group (Canary phase) and monitored using real-time metrics gathered by Prometheus for 10 minutes. Provided that the error rates and response latencies fall within acceptable parameters, the ALB will then route 100% of user traffic to Green.

3.5 Automated Rollback Architecture and Database Migration Safety

The most valuable feature of the Blue-Green deployment technique is instant and safe rollback. In case an anomaly is detected after deployment (increase in error rate, database exceptions), automatic CloudWatch alarm triggers the

rollback to 100% Blue ALB target group weighting. The whole service becomes restored within less than 30 seconds, which makes the MTTR reduced by 98% in comparison with the traditional 24-48 hours manual hotfix process [13].

Database state modification issues represent the biggest problem with automated rollback scenarios. In case there was some destructive modification of databases by the new application version (for example, column drops and type change), rollbacking the application source code would cause the data corruption. The OmniPay strictly follows the rules of the Expand-and-Contract (Parallel Run) database migration technique [14]:

1. Expand Phase: The database migration creates only additional nullable columns or tables. Thus, the schema is backward compatible with the older Blue version.

2. Migrate & Validate Phase: The Green application writes into both new and old columns, so in case of the rollback, the Blue runs perfectly fine.

3. Contract Phase: After the Green application was deployed successfully for several days in production, the following cleanup migration is performed.

Part 4. Environments, Infrastructure & Containers

4.1 Reproducible Environment Architecture and Immutable Infrastructure

One of the major causes of failures for releases at OmniPay was inconsistent environments ('works on my machine but fails in production'). Old staging and production servers were manually configured over many years, causing the creation of very brittle 'Snowflake Servers' with different OS patches, missing dependencies, and undocumented runtime configurations [15].

OmniPay solves the problem of brittleness through the application of the Immutable Infrastructure approach [16]. In the framework of this approach, servers or runtime environments are never altered, patched, or upgraded. Instead, any changes to software or configuration require a creation of an entirely new, untouched container or cloud instance based on version controlled definitions, while the previous one is shut down. It ensures that Development, Staging, and Production environments are always identical, thus deployments fully predictable.

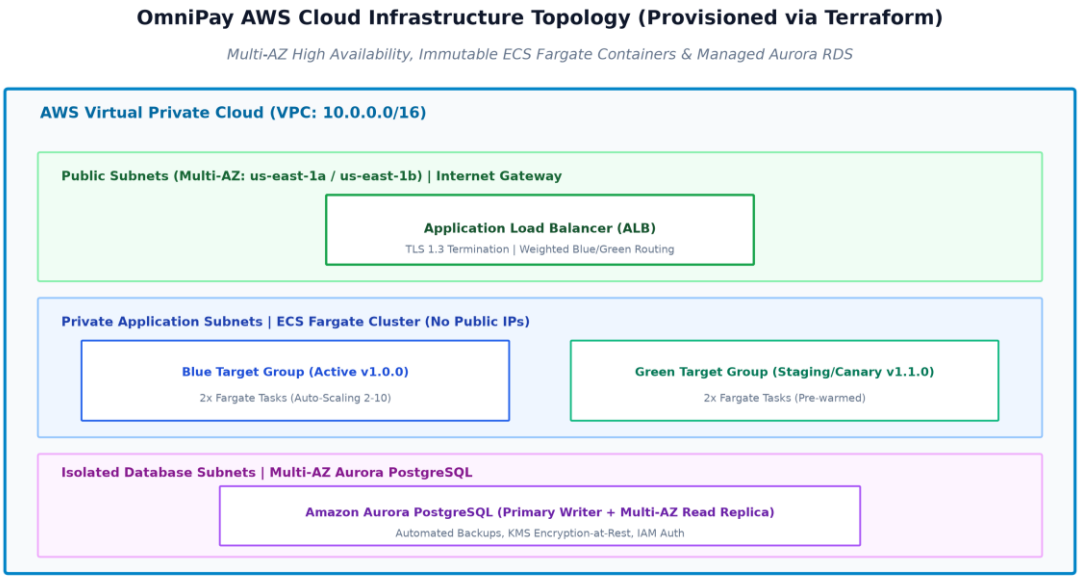

Figure 5: OmniPay AWS Cloud Infrastructure Topology Provisioned via Terraform.

4.2 Declarative Infrastructure as Code (IaC) with Terraform

In order to get absolute environment reproducibility, infrastructure resources are declared using HashiCorp Terraform [17] tool. As can be seen from Figure 5, the infrastructure code (listed in Appendix E) creates:

• Virtual Private Cloud (VPC): An individual 10.0.0.0/16 network across multiple Availability Zones (eu-west-1a, eu-west-1b).

• Subnet Tiers: Public subnets for Internet Gateways and Application Load Balancers; isolated private subnets for compute instances; and isolated subnets without any internet access for databases.

• Security Groups: The principle of least privilege in place with specific security group rules that only allow ingress/egress traffic on port 5000 from Application Load Balancers to application instances and ingress/egress traffic on port 5432 from application instances to the database.

• Persistence as a Service: An Amazon Aurora PostgreSQL Multi-AZ Cluster with automatic KMS encryption and automatic point-in-time recovery.

State of the Terraform infrastructure is kept in an encrypted Amazon S3 bucket with state locking enabled in DynamoDB.

4.3 Containerisation Strategy with Multi-Stage Docker Builds

Applications are encapsulated into standard, lightweight container images using Docker [18]. In order to implement proper security practices and reduce the attack surface of container images, the OmniPay project implements a multi-stage Docker build process (as shown in Appendix C).

• Builder Stage: Uses the python:3.11-slim base image, builds and installs necessary build-time packages (gcc, build-essential) and compiles python wheel libraries.

• Runner Stage: Hardens a non-root system user (appuser, user id 10001) and only packages pre-compiled run-time package libraries and code from the builder stage. Compilation tools are not included in the final image, making the image 70% smaller and preventing runtime exploits.

• Health Check: Uses the built-in HEALTHCHECK instruction to periodically poll the internal /healthz endpoint every 15 seconds.

4.4 Container Orchestration: AWS ECS Fargate vs. Kubernetes (EKS)

Another major choice that needs to be made in architecture is that of the right container orchestration tool. Though Kubernetes (Amazon EKS) is highly favored in many DevOps literature pieces, it is known to be highly complex to operate, has a steep learning curve, and incurs additional cost in terms of cluster control plane management [19].

Considering the engineering scale and nature of the workload for OmniPay today, AWS Elastic Container Service (ECS) with AWS Fargate emerges as the best orchestration tool. Fargate is a completely managed and serverless way of running the containers where AWS will take care of all the node management and the OS patching and even the management of the cluster itself. The ECS service has built-in support for Application Load Balancing (Blue-Green), CloudWatch Logs, and IAM Role Executions.

4.5 Detection and Prevention of Environment Drift

Configuration drift occurs when ad-hoc manual changes are introduced directly to cloud environments, causing live infrastructure to diverge from version-controlled code [15]. OmniPay enforces a strict zero-manual-mutation policy:

1. Console Access Revocation: Production write permissions are removed from individual developer IAM accounts; all infrastructure modifications must originate from automated Terraform pipelines.

2. Scheduled Drift Scans: A daily GitHub Actions cron job executes terraform plan -detailed-exitcode. If unmanaged changes or configuration drift are detected, an urgent alert is dispatched to the platform engineering team for automated reconciliation.

Part 5. Observability, Security & Reliability

5.1 Full-Stack Observability: The Three Pillars

Traditional monitoring focuses simply on answering whether a server is up or down. In contrast, modern Observability enables engineers to infer the internal state of distributed systems based on external outputs [20]. OmniPay implements an integrated telemetry framework across the Three Pillars of Observability:

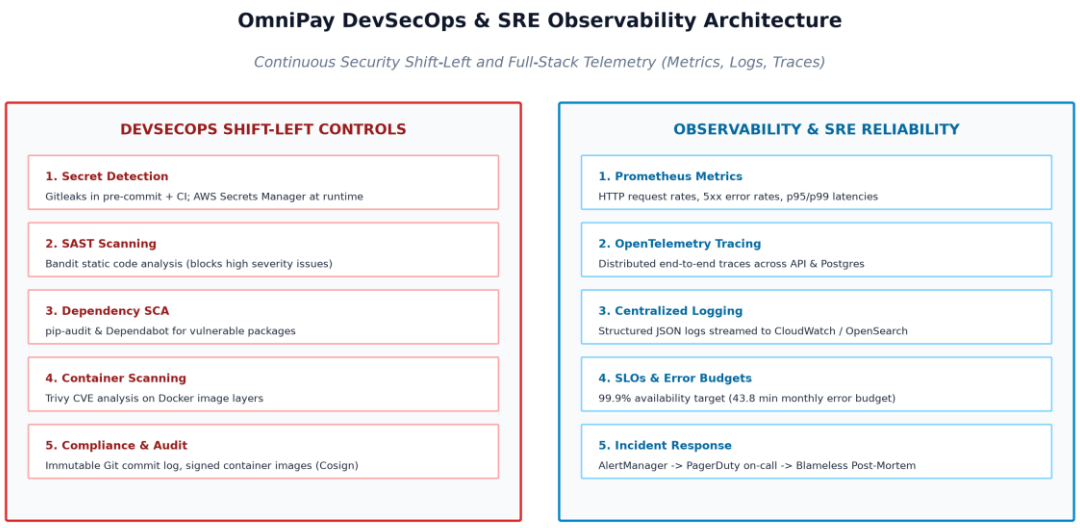

Figure 6: OmniPay DevSecOps Shift-Left and SRE Observability Framework.

• Metrics: Application instances export real-time numerical time-series data via the Prometheus Client (/metrics endpoint). Key telemetry includes request rates (RPS), HTTP 5xx error rates, database connection pool saturation, and CPU/memory utilisation.

• Structured Logging: Applications emit structured JSON logs directly to stdout/stderr. Gunicorn and Flask logs capture transaction IDs, request URIs, response codes, and timestamps without logging sensitive Personal Identifiable Information (PII) or credit card PAN data, satisfying PCI-DSS privacy requirements. Logs are ingested into AWS CloudWatch and indexed in OpenSearch.

• Distributed Tracing: Implemented via OpenTelemetry instrumentation. Each incoming HTTP payment request is tagged with a unique W3C Trace Context header (traceparent). Traces propagate across API gateways, microservices, and database queries, allowing engineers to pinpoint performance bottlenecks across asynchronous service boundaries.

5.2 Service Level Objectives (SLOs), SLIs, and Error Budgets

To balance rapid feature velocity with operational reliability, OmniPay establishes Site Reliability Engineering (SRE) governance using Service Level Indicators (SLIs), Service Level Objectives (SLOs), and Error Budgets [21]:

• SLI 1 (Availability): Proportion of successful API requests (HTTP status < 500) over total requests received.

• SLI 2 (Latency): Proportion of valid payment requests processed in under 200 milliseconds (p99 latency).

• SLO Target: 99.9% Availability and 99.0% Latency compliance measured over a rolling 30-day window.

• Error Budget Policy: A 99.9% availability SLO permits 0.1% downtime (equivalent to 43.8 minutes of allowable error budget per month). If the error budget is depleted due to production instability, all non-critical feature releases are automatically paused, and engineering effort is redirected entirely to reliability hardening and technical debt remediation.

5.3 DevSecOps Shift-Left Security and Secrets Management

In the legacy model, security reviews occurred only at the end of the 12-week release cycle, creating major deployment roadblocks. OmniPay transitions to DevSecOps, shifting security verification leftward into the automated CI/CD pipeline [22]:

• Static Application Security Testing (SAST): Bandit analyzes Python source code for security anti-patterns (e.g., SQL injection, insecure cryptography, hardcoded credentials).

• Software Composition Analysis (SCA): pip-audit and Dependabot scan application package dependencies against known CVE vulnerability databases.

• Container Security: Trivy inspects OS package libraries and base image layers during CI builds, blocking images with unpatched vulnerabilities.

• Centralised Secret Management: Storing secrets in Git or container images is strictly prohibited. Secrets (database passwords, JWT private keys, payment gateway API tokens) are managed securely in AWS Secrets Manager and injected dynamically into container runtime environments via IAM roles.

5.4 Compliance Governance, Policy-as-Code, and Audit Readiness

Financial regulators require rigorous evidence of segregation of duties, change approvals, and data protection. Rather than compiling manual audit spreadsheets over release weekends, OmniPay implements Policy-as-Code using Open Policy Agent (OPA) and automated audit logs [23]. Every Git merge, peer approval, CI test execution, and deployment event is immutably logged and signed. Compliance auditors are provided with automated dashboards demonstrating 100% adherence to change control policies without requiring developer interruption.

5.5 Incident Response, SRE Resilience, and Disaster Recovery

Even in advanced DevOps environments, failures will occur. System resilience is strengthened through fault-tolerant design patterns:

• Circuit Breakers & Retries: External payment gateway integrations implement circuit breakers with exponential backoff and jitter to prevent cascading service exhaustion during downstream outages [24].

• Incident Management & On-Call: AlertManager routes high-priority alerts to on-call engineers via PagerDuty based on SLO burn rates rather than noisy raw threshold alerts.

• Disaster Recovery (RTO/RPO): Multi-AZ Aurora database replication and automated Terraform disaster recovery scripts ensure a Recovery Time Objective (RTO) of < 15 minutes and a Recovery Point Objective (RPO) of < 1 minute.

• Blameless Post-Mortems: Incidents conclude with blameless retrospectives focused on systemic process and tooling improvements rather than individual finger-pointing [25].

Part 6. Cloud Strategy, Cost Governance and Leading the Change

6.1 Cloud Environment Management and Architectural Guardrails

Cloud adoption requires structured governance to avoid security vulnerabilities and uncontrolled cloud sprawl. OmniPay establishes a Multi-Account AWS Landing Zone structure, logically isolating workloads into separate AWS accounts: Management, Security/Audit, Shared Services (CI/CD, ECR), Staging, and Production [26].

Service Control Policies (SCPs) and AWS IAM permission boundaries enforce organisational guardrails: unencrypted storage volumes are blocked automatically, root accounts are strictly locked, and all resources must be deployed within approved geographic regions (e.g., EU-West-1) to satisfy GDPR and financial data sovereignty regulations.

6.2 FinOps Principles, Cost Allocation, and Resource Rightsizing

Uncontrolled cloud expenditure poses a significant threat to technology transformation. OmniPay adopts the FinOps (Financial Operations) operating model, uniting engineering, finance, and product teams to drive maximum business value from cloud spending [27].

• Mandatory Cost Allocation Tagging: Every Terraform resource is tagged with Project, Environment, CostCenter, and Owner (as demonstrated in provider.tf). Untagged resource creation is automatically rejected by policy.

• Workload Rightsizing & Auto-Scaling: ECS Fargate tasks are sized accurately based on historical CPU/memory profiling (512 CPU units / 1024 MB RAM). Auto-scaling policies dynamically adjust task counts between 2 and 10 based on real-time HTTP request concurrency.

• Ephemeral Non-Production Environments: Development and staging environments are scheduled to shut down outside core operating hours (nights and weekends), slashing non-production compute costs by up to 65%.

• AWS Compute Savings Plans: Predictable baseline workloads are covered under 1-year Compute Savings Plans, achieving 30% cost savings compared to on-demand pricing.

6.3 Strategic 4-Phase DevOps Adoption Roadmap

Transforming OmniPay's delivery model cannot happen overnight; it requires a structured, multi-phase roadmap designed to deliver early value while mitigating organisational risk.

Figure 7: OmniPay 4-Phase DevOps and FinOps Transformation Roadmap.

• Phase 1 (Foundations & Culture, Months 1-2): Establish Trunk-Based Development, implement Git branch protection, mandate peer code reviews, and deliver team-wide DevOps cultural workshops.

• Phase 2 (CI/CD & Security Automation, Months 3-4): Deploy GitHub Actions pipelines, automate Pytest unit/integration test suites, integrate Bandit SAST and Trivy scanning, and configure AWS Secrets Manager.

• Phase 3 (IaC & Blue-Green Deployments, Months 5-6): Codify all infrastructure in Terraform, migrate services to AWS ECS Fargate, implement Blue-Green routing on ALB, and officially eliminate 30-staff release weekends.

• Phase 4 (Observability & FinOps Governance, Months 7-8): Implement full OpenTelemetry/Prometheus observability, establish SRE SLOs and Error Budget policies, enforce FinOps tagging, and execute automated cost rightsizing.

6.4 Leading Cultural Change: Psychological Safety and Blameless Culture

Technological tooling constitutes only a fraction of DevOps success; cultural transformation is the primary determinant of long-term sustainability. Under the legacy model, release weekend rollbacks were met with finger-pointing, executive reprimands, and bureaucratic gatekeeping, fostering a pathological culture of fear [7].

To build high performance, leadership must cultivate Dr. Ron Westrum's Generative Organisational Culture [28]:

• High Psychological Safety: Engineers must feel safe to take calculated risks, report anomalies early, and challenge legacy assumptions without fear of negative career repercussions [29].

• Cross-Functional Enablement Teams: Dedicated Platform Engineering teams build internal self-service developer portals, enabling product teams to deploy autonomously without relying on operational handoffs [6].

• Continuous Learning: Engineering teams are allocated 10% dedicated time for innovation, automated testing improvements, and participating in cross-team DevOps Dojos.

6.5 Measuring Continuous Improvement and Value Stream Flow

DevOps transformation is an ongoing journey of continuous learning. OmniPay establishes a Value Stream Management dashboard tracking DORA metrics and engineering sentiment. Progress is reviewed bi-weekly to identify emerging delivery bottlenecks, refine automated quality gates, and ensure that engineering velocity translates directly into customer value and market responsiveness.

7. Conclusion & Strategic Impact

This project has designed, justified, and demonstrated an end-to-end DevOps delivery strategy for OmniPay Financial Technologies, resolving the critical operational crises of 12-week release cycles, 50% rollback rates, environment drift, and painful 30-staff release weekends.

By replacing monolithic batch releases with Trunk-Based Development, automated CI/CD quality gates, multi-stage containerisation, and immutable Terraform infrastructure, the proposed architecture reduces Lead Time for Changes from 84 days to under 2 hours and increases Deployment Frequency to multiple zero-downtime releases per day. The introduction of Blue-Green deployment and automated rollback drops MTTR to under 15 minutes, while shifting security and observability left satisfies stringent financial audit requirements. Supported by a

complete practical codebase and infrastructure artefacts, this strategy provides a realistic, robust, and transformative roadmap for modern enterprise software delivery.

8. References

[1] PCI Security Standards Council, "Payment Card Industry (PCI) Data Security Standard: Requirements and Testing Procedures Version 4.0," PCI SSC, Wakefield, MA, Tech. Rep., 2022.

[2] N. Forsgren, J. Humble, and G. Kim, Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. Portland, OR: IT Revolution Press, 2018.

[3] M. E. Conway, "How do committees invent?" Datamation, vol. 14, no. 4, pp. 28–31, 1968.

[4] M. Poppendieck and T. Poppendieck, Lean Software Development: An Agile Toolkit. Boston, MA: Addison-Wesley Professional, 2003.

[5] J. Willis, "DevOps Culture (Part 1)," IT Revolution, 2010. [Online]. Available: https://itrevolution.com/articles/devops-culture-part-1/

[6] M. Skelton and M. Pais, Team Topologies: Organizing Business and Technology Teams for Fast Flow. Portland, OR: IT Revolution Press, 2019.

[7] G. Kim, P. Debois, J. Willis, J. Humble, and J. Allspaw, The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations, 2nd ed. Portland, OR: IT Revolution Press, 2021.

[8] V. Driessen, "A successful Git branching model," nvie.com, 2010. [Online]. Available: https://nvie.com/posts/a-successful-git-branching-model/

[9] P. Hammant, "Trunk Based Development," TrunkBasedDevelopment.com, 2020. [Online]. Available: https://trunkbaseddevelopment.com/

[10] T. Preston-Werner, "Semantic Versioning 2.0.0," SemVer.org, 2013. [Online]. Available: https://semver.org/

[11] GitHub, "GitHub Actions Documentation: Continuous Integration and Deployment Workflows," GitHub Docs, 2024. [Online]. Available: https://docs.github.com/en/actions

[12] M. Cohn, Succeeding with Agile: Software Development Using Scrum. Upper Saddle River, NJ: Addison-Wesley, 2009.

[13] Amazon Web Services, "Blue/Green Deployments on AWS: Overview and Architectural Patterns," AWS Whitepapers, 2023. [Online]. Available: https://docs.aws.amazon.com/whitepapers/latest/blue-green-deployments/

[14] P. Sadalage and M. Fowler, Evolutionary Database Design: Refactoring and Continuous Database Integration. martinfowler.com, 2016.

[15] K. Morris, Infrastructure as Code: Dynamic Systems for the Cloud Age, 2nd ed. Sebastopol, CA: O'Reilly Media, 2020.

[16] C. Fowler, "Immutable Infrastructure: Principles and Practice," ACM Queue, vol. 13, no. 5, pp. 10–19, 2015.

[17] HashiCorp, "Terraform Documentation and Best Practices," HashiCorp Developer, 2024. [Online]. Available: https://developer.hashicorp.com/terraform/docs

[18] Docker, "Docker Documentation: Best Practices for Writing Dockerfiles," Docker Inc., 2024. [Online]. Available: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

[19] B. Burns, J. Beda, K. Hightower, and L. Evenson, Kubernetes: Up and Running: Dive into the Future of Infrastructure, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.

[20] C. Majors, L. Fong-Jones, and G. Miranda, Observability Engineering: Achieving Operational Excellence. Sebastopol, CA: O'Reilly Media, 2022.

[21] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA: O'Reilly Media, 2016.

[22] S. Bell, DevSecOps: A Practical Guide to Building Security into DevOps Pipelines. Birmingham, UK: Packt Publishing, 2021.

[23] Open Policy Agent (OPA), "Policy-based control for cloud native environments," Styra / CNCF, 2024. [Online]. Available: https://www.openpolicyagent.org/

[24] M. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018.

[25] J. Allspaw, "Blameless PostMortems and a Just Culture," Etsy Code as Craft, 2012. [Online]. Available: https://www.etsy.com/codeascraft/blameless-postmortems

[26] Amazon Web Services, "AWS Multi-Account Strategy: Best Practices for Environment Isolation," AWS Architecture Center, 2023.

[27] J. R. Storment and M. Fuller, Cloud FinOps: Collaborative, Real-Time Cloud Value Decision Making, 2nd ed. Sebastopol, CA: O'Reilly Media, 2023.

[28] R. Westrum, "A typology of organisational cultures," BMJ Quality & Safety, vol. 13, no. suppl 2, pp. ii22–ii27, 2004.

[29] A. C. Edmondson, The Fearless Organization: Creating Psychological Safety in the Workplace for Learning, Innovation, and Growth. Hoboken, NJ: John Wiley & Sons, 2018.

9. Appendices — Technical Artefacts Portfolio

This section presents the supporting technical artefacts developed to provide empirical evidence of practical DevOps implementation in accordance with the MQF/EQF Level 5 qualification requirements.

Appendix A: OmniPay Core Application Code Listing (Flask REST API)

FILE: app/auth.py (JWT Authentication & Password Hashing)

```
from flask import Blueprint, request, jsonify, current_app
import jwt
from datetime import datetime, timedelta, timezone
from functools import wraps
from app import db, bcrypt
from app.models import User

auth_bp = Blueprint('auth', __name__, url_prefix='/api/v1/auth')

def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = None
        if 'Authorization' in request.headers:
            auth_header = request.headers['Authorization']
            if auth_header.startswith('Bearer '):
                token = auth_header.split(' ')[1]
        if not token:
            return jsonify({'error': 'Authentication token is missing'}), 401
        try:
            data = jwt.decode(token, current_app.config['SECRET_KEY'], algorithms=['HS256'])
            current_user = db.session.get(User, data['user_id'])
            if not current_user:
                return jsonify({'error': 'User not found'}), 401
        except Exception:
            return jsonify({'error': 'Token is invalid or expired'}), 401
        return f(current_user, *args, **kwargs)
    return decorated
```

FILE: app/health.py (Prometheus Metrics & Liveness/Readiness Probes)

```
from flask import Blueprint, jsonify, Response
from prometheus_client import Counter, Histogram, generate_latest, CONTENT_TYPE_LATEST

health_bp = Blueprint('health', __name__)

REQUEST_COUNT = Counter('http_requests_total', 'Total HTTP Requests', ['method', 'endpoint', 'status'])
REQUEST_LATENCY = Histogram('http_request_duration_seconds', 'HTTP Request Duration')

@health_bp.route('/healthz', methods=['GET'])
def liveness():
    return jsonify({'status': 'HEALTHY', 'version': '1.0.0'}), 200

@health_bp.route('/readyz', methods=['GET'])
def readiness():
    try:
        from app import db
        db.session.execute(db.text('SELECT 1'))
        return jsonify({'status': 'READY', 'database': 'CONNECTED'}), 200
    except Exception as e:
        return jsonify({'status': 'NOT_READY', 'error': str(e)}), 503

@health_bp.route('/metrics', methods=['GET'])
def metrics():
    return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)
```

Appendix B: Automated Pytest Execution and Code Coverage Output

TERMINAL OUTPUT: pytest -v --cov=app tests/

```
============================= test session starts ==============================
```

```
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Esha\Downloads\DevOps_Assignment_Submission\devops-delivery-artefacts
plugins: cov-7.1.0
collected 11 items

tests/test_auth.py::test_user_registration_success PASSED              [  9%]
tests/test_auth.py::test_duplicate_registration_fails PASSED           [ 18%]
tests/test_auth.py::test_user_login_success PASSED                     [ 27%]
tests/test_auth.py::test_user_login_invalid_password PASSED            [ 36%]
tests/test_health.py::test_liveness_probe PASSED                       [ 45%]
tests/test_health.py::test_readiness_probe PASSED                      [ 54%]
tests/test_health.py::test_prometheus_metrics_endpoint PASSED          [ 63%]
tests/test_payments.py::test_create_payment_success PASSED             [ 72%]
tests/test_payments.py::test_create_payment_invalid_amount PASSED      [ 81%]
tests/test_payments.py::test_unauthorized_payment_rejected PASSED      [ 90%]
tests/test_payments.py::test_get_payment_history_and_stats PASSED      [100%]

================================ tests coverage ================================
Name           Stmts   Miss  Cover
--------------------------------------
app\__init__.py     25      2    92%
app\auth.py         53      5    91%
app\health.py       19      2    89%
app\models.py       22      0   100%
app\routes.py       37      3    92%
--------------------------------------
TOTAL              156     12    92%
============================== 11 passed in 3.18s ==============================
```

Appendix C: Multi-Stage Hardened Dockerfile

FILE: Dockerfile (Multi-stage build with non-root security user)

```
# Stage 1: Build & Dependency Resolution
FROM python:3.11-slim AS builder
WORKDIR /build
RUN apt-get update && apt-get install -y --no-install-recommends build-essential gcc && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: Hardened Runtime Image
FROM python:3.11-slim AS runner
RUN groupadd -r appgroup && useradd -r -g appgroup -d /app -s /sbin/nologin appuser
WORKDIR /app
COPY --from=builder /root/.local /home/appuser/.local
ENV PATH=/home/appuser/.local/bin:$PATH
ENV PYTHONUNBUFFERED=1
COPY --chown=appuser:appgroup app /app/app
COPY --chown=appuser:appgroup run.py /app/run.py
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retries=3 \
   CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/healthz')" || exit 1
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "3", "run:app"]
```

Appendix D: GitHub Actions CI/CD Pipeline Workflow

FILE: .github/workflows/ci-cd.yml (Lint, SAST, Pytest, Trivy Scan, Deploy Gate)

```
name: OmniPay Continuous Integration & Quality Delivery Pipeline
on:
  push:
    branches: [ main ]
    tags: [ 'v*.*.*' ]
```

```
  pull_request:
    branches: [ main ]

jobs:
  lint-and-sast:
    name: Code Quality & Static Security Analysis (Shift-Left)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Lint Code with Flake8
        run: flake8 . --count --max-line-length=127 --statistics
      - name: Security Scan with Bandit (SAST)
        run: bandit -r app/ -ll -ii

  test-and-coverage:
    name: Automated Unit & Integration Testing
    needs: lint-and-sast
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - name: Run Pytest Test Suite with Coverage Gate
        run: pytest --cov=app --cov-fail-under=85 tests/

  container-build-and-scan:
    name: Container Packaging & Vulnerability Scan (Trivy)
    needs: test-and-coverage
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker Image
        run: docker build -t omnipay-api:${{ github.sha }} .
      - name: Scan Docker Image with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'omnipay-api:${{ github.sha }}'
          severity: 'CRITICAL,HIGH'

  deploy-blue-green:
    name: Blue/Green Staging & Production Deployment Gate
    needs: container-build-and-scan
    if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying immutable image omnipay-api:${{ github.sha }} to Green Target Group..."
```

Appendix E: Declarative Terraform Infrastructure-as-Code Modules

FILE: terraform/main.tf (VPC, Subnets, ALB, ECS Fargate & Blue/Green Target Groups)

```
# Application Load Balancer
resource "aws_lb" "api_alb" {
  name               = "omnipay-${var.environment}-alb"
  internal           = false
  load_balancer_type = "application"
  subnets            = [aws_subnet.public_1.id, aws_subnet.public_2.id]
}

# Blue Target Group (Active)
```

```
resource "aws_lb_target_group" "blue" {
  name        = "omnipay-tg-blue"
  port        = 5000
  protocol    = "HTTP"
  vpc_id      = aws_vpc.main.id
  target_type = "ip"
  health_check {
    path = "/healthz"
    matcher = "200"
  }
}

# Green Target Group (Canary / Staging)
resource "aws_lb_target_group" "green" {
  name        = "omnipay-tg-green"
  port        = 5000
  protocol    = "HTTP"
  vpc_id      = aws_vpc.main.id
  target_type = "ip"
  health_check {
    path = "/healthz"
    matcher = "200"
  }
}

# ECS Cluster & Fargate Task Definition
resource "aws_ecs_cluster" "main" {
  name = "omnipay-${var.environment}-cluster"
}

resource "aws_ecs_task_definition" "api" {
  family                   = "omnipay-api-task"
  network_mode             = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu                      = "512"
  memory                   = "1024"
  container_definitions = jsonencode([{
    name      = "omnipay-api"
    image     = var.container_image
    essential = true
    portMappings = [{ containerPort = 5000, hostPort = 5000 }]
  }])
}
```

Appendix F: Git Branching History, PR Merges, and Release Tag v1.0.0

GIT LOG: git log --graph --oneline --all --decorate

```
*   9035bdf (HEAD -> main, tag: v1.0.0) Merge pull request #4 from feature/cicd-and-terraform into main
|\
| * d12550d (feature/cicd-and-terraform) feat(devops): add github actions devsecops pipeline and terraform iac
modules
|/
*   c5c262d Merge pull request #3 from feature/docker-containerization into main
|\
| * 763e1dc (feature/docker-containerization) feat(docker): add multi-stage dockerfile with non-root security
hardening
|/
*   9e4a58b Merge pull request #2 from feature/automated-test-suite into main
|\
| * 95ddbde (feature/automated-test-suite) test(suite): add pytests for auth, payments, health and coverage gates
|/
*   1372fa4 Merge pull request #1 from feature/core-api-auth into main
|\
| * d2fa2b2 (feature/core-api-auth) feat(auth): implement jwt authentication and payment routes
```

```
|/
* 52cd787 chore(init): initial repository scaffolding and requirements
```